

NT-C-016 | C Programming | Course Knowledge Profile

Identity, purpose, prerequisites, scope and boundaries

Category	Programming Foundations
Target learners	Beginners needing procedural programming and memory fundamentals.
Prerequisites	Basic computer use and logical thinking.
Approved duration	6-8 weeks / 60-90 guided hours.
Final outcome	Develop and debug modular C programs using arrays, pointers, structures and files.
Assessment profile	Programming/Platform

Approved tools / platforms

C, GCC or compatible compiler, standard library, debugger, Make basics, Git.

Knowledge boundaries

Include	Quality rule	Exclude / escalate
The eight approved modules, four sections, named tools, projects and mapped entry-level roles.	Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.	Do not treat copied code or syntax recall as proof of implementation skill.
Current product/platform procedures only when version and date are known.	Use primary documentation and record the source version.	Do not invent current commands, prices, tax rates, policies or certification status.
Evidence-based career guidance.	Cite tests, projects, capstone, viva and verified resume evidence.	No guaranteed job, placement, salary, interview or employer claim.

LLM answer pattern: identify the course and version, answer from approved records, cite record IDs, distinguish verified facts from guidance, and state any missing evidence.

NT-C-016 | C Programming | NT-C-016-S01 Deep Knowledge

Modules 1-2 | objectives, concepts, evidence and remediation

Section competency	Compiler, variables, types and I/O; Conditions, loops and operators
Completion evidence	Basic programs; Logic exercises
Section weight	20% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 1 - Environment and syntax

Objective: Compiler, variables, types and I/O

Observable evidence: Basic programs

Concept	Approved knowledge statement	Application behaviour	Evidence
Compiler	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when compiler is appropriate; apply it in a C Programming task; verify the result.	Basic programs
variables	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when variables is appropriate; apply it in a C Programming task; verify the result.	Basic programs
types	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when types is appropriate; apply it in a C Programming task; verify the result.	Basic programs
I/O	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when i/o is appropriate; apply it in a C Programming task; verify the result.	Basic programs

Module 2 - Control flow

Objective: Conditions, loops and operators

Observable evidence: Logic exercises

Concept	Approved knowledge statement	Application behaviour	Evidence
Conditions	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when conditions is appropriate; apply it in a C Programming task; verify the result.	Logic exercises
loops	A design approach that organises behaviour and data around objects and clear responsibilities. In C Programming, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when loops is appropriate; apply it in a C Programming task; verify the result.	Logic exercises
operators	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when operators is appropriate; apply it in a C Programming task; verify the result.	Logic exercises

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Basic programs; Logic exercises, return to Modules 1-2, complete one guided practice and then use a new equivalent test form.

NT-C-016 | C Programming | NT-C-016-S02 Deep Knowledge

Modules 3-4 | objectives, concepts, evidence and remediation

Section competency	Parameters, return values, scope and headers; Memory layout and text processing
Completion evidence	Modular program; Array/string utilities
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 3 - Functions

Objective: Parameters, return values, scope and headers

Observable evidence: Modular program

Concept	Approved knowledge statement	Application behaviour	Evidence
Parameters	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when parameters is appropriate; apply it in a C Programming task; verify the result.	Modular program
return values	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when return values is appropriate; apply it in a C Programming task; verify the result.	Modular program
scope	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when scope is appropriate; apply it in a C Programming task; verify the result.	Modular program
headers	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when headers is appropriate; apply it in a C Programming task; verify the result.	Modular program

Module 4 - Arrays and strings

Objective: Memory layout and text processing

Observable evidence: Array/string utilities

Concept	Approved knowledge statement	Application behaviour	Evidence
Memory layout	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when memory layout is appropriate; apply it in a C Programming task; verify the result.	Array/string utilities
text processing	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when text processing is appropriate; apply it in a C Programming task; verify the result.	Array/string utilities

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Modular program; Array/string utilities, return to Modules 3-4, complete one guided practice and then use a new equivalent test form.

NT-C-016 | C Programming | NT-C-016-S03 Deep Knowledge

Modules 5-6 | objectives, concepts, evidence and remediation

Section competency	Addresses, pointers and dynamic memory; Structs, enums and data organisation
Completion evidence	Memory exercises; Record application
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 5 - Pointers

Objective: Addresses, pointers and dynamic memory

Observable evidence: Memory exercises

Concept	Approved knowledge statement	Application behaviour	Evidence
Addresses	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when addresses is appropriate; apply it in a C Programming task; verify the result.	Memory exercises
pointers	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when pointers is appropriate; apply it in a C Programming task; verify the result.	Memory exercises
dynamic memory	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when dynamic memory is appropriate; apply it in a C Programming task; verify the result.	Memory exercises

Module 6 - Structures

Objective: Structs, enums and data organisation

Observable evidence: Record application

Concept	Approved knowledge statement	Application behaviour	Evidence
Structs	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when structs is appropriate; apply it in a C Programming task; verify the result.	Record application
enums	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when enums is appropriate; apply it in a C Programming task; verify the result.	Record application
data organisation	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when data organisation is appropriate; apply it in a C Programming task; verify the result.	Record application

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Memory exercises; Record application, return to Modules 5-6, complete one guided practice and then use a new equivalent test form.

NT-C-016 | C Programming | NT-C-016-S04 Deep Knowledge

Modules 7-8 | objectives, concepts, evidence and remediation

Section competency	File I/O, validation and return codes; Modular design, testing and debugging
Completion evidence	Persistent utility; Completed C project
Section weight	30% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 7 - Files and errors

Objective: File I/O, validation and return codes

Observable evidence: Persistent utility

Concept	Approved knowledge statement	Application behaviour	Evidence
File I/O	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when file i/o is appropriate; apply it in a C Programming task; verify the result.	Persistent utility
validation	Checking input or output against defined rules before trusting it. In C Programming, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when validation is appropriate; apply it in a C Programming task; verify the result.	Persistent utility
return codes	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when return codes is appropriate; apply it in a C Programming task; verify the result.	Persistent utility

Module 8 - Debugging capstone

Objective: Modular design, testing and debugging

Observable evidence: Completed C project

Concept	Approved knowledge statement	Application behaviour	Evidence
Modular design	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when modular design is appropriate; apply it in a C Programming task; verify the result.	Completed C project
testing	Systematic verification that expected behaviour and important failure cases work correctly. In C Programming, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when testing is appropriate; apply it in a C Programming task; verify the result.	Completed C project
debugging	A defined competency within C Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when debugging is appropriate; apply it in a C Programming task; verify the result.	Completed C project

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Persistent utility; Completed C project, return to Modules 7-8, complete one guided practice and then use a new equivalent test form.

NT-C-016 | C Programming | Skill Ontology & Dependency Map

Atomic skills, learning order and evidence strength

Skill ID	Competency	Depends on	Evidence	Type
NT-C-016-SK01	Environment and syntax	Prerequisite foundation	Basic programs	Critical
NT-C-016-SK02	Control flow	M01 plus earlier foundations	Logic exercises	Critical
NT-C-016-SK03	Functions	M02 plus earlier foundations	Modular program	Supporting
NT-C-016-SK04	Arrays and strings	M03 plus earlier foundations	Array/string utilities	Supporting
NT-C-016-SK05	Pointers	M04 plus earlier foundations	Memory exercises	Supporting
NT-C-016-SK06	Structures	M05 plus earlier foundations	Record application	Critical
NT-C-016-SK07	Files and errors	M06 plus earlier foundations	Persistent utility	Supporting
NT-C-016-SK08	Debugging capstone	M07 plus earlier foundations	Completed C project	Critical

Evidence-strength hierarchy

Strength	Evidence source	Use
High	Trainer-observed practical, viva, working source/project files and reproducible output.	May support verified mastery when rubric threshold is met.
Medium	Scored mock tests, reviewed assignments and documented portfolio artifacts.	Supports mastery with corroboration.
Low	Resume claim, self-report, attendance or video completion without demonstration.	May trigger verification; cannot alone support Apply Now.

CareerTours must not treat attendance, time spent or content opened as skill mastery.

NT-C-016 | C Programming | Assessment Knowledge & Sample Items

Non-live examples for LLM training and evaluation

Sample ID	Type	Question	Approved answer / rubric anchor
NT-C-016-S01-EX01	Evidence/application	What evidence best demonstrates the combined competency of Environment and syntax and Control flow?	The learner should provide Basic programs; Logic exercises and explain how the work applies Compiler, variables, types and I/O; Conditions, loops and operators. Recall alone is insufficient.
NT-C-016-S01-EX02	Scenario/remediation	A learner reports completing Control flow but cannot produce Logic exercises. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-016-S01, request Logic exercises, and allow a new equivalent test only after remediation.
NT-C-016-S02-EX01	Evidence/application	What evidence best demonstrates the combined competency of Functions and Arrays and strings?	The learner should provide Modular program; Array/string utilities and explain how the work applies Parameters, return values, scope and headers; Memory layout and text processing. Recall alone is insufficient.
NT-C-016-S02-EX02	Scenario/remediation	A learner reports completing Arrays and strings but cannot produce Array/string utilities. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-016-S02, request Array/string utilities, and allow a new equivalent test only after remediation.
NT-C-016-S03-EX01	Evidence/application	What evidence best demonstrates the combined competency of Pointers and Structures?	The learner should provide Memory exercises; Record application and explain how the work applies Addresses, pointers and dynamic memory; Structs, enums and data organisation. Recall alone is insufficient.
NT-C-016-S03-EX02	Scenario/remediation	A learner reports completing Structures but cannot produce Record application. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-016-S03, request Record application, and allow a new equivalent test only after remediation.
NT-C-016-S04-EX01	Evidence/application	What evidence best demonstrates the combined competency of Files and errors and Debugging capstone?	The learner should provide Persistent utility; Completed C project and explain how the work applies File I/O, validation and return codes; Modular design, testing and debugging. Recall alone is insufficient.
NT-C-016-S04-EX02	Scenario/remediation	A learner reports completing Debugging capstone but cannot produce Completed C project. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-016-S04, request Completed C project, and allow a new equivalent test only after remediation.

Live bank rule: Generate at least three times the live quantity for each section using the approved Programming/Platform blueprint. Human-review every scored item before initial production release.

NT-C-016 | C Programming | Labs, Projects & Scoring Rubrics

Practical proof of learning and ownership

Level	Approved project	Skills evidenced	Required deliverables
Mini 1	Calculator toolkit	Functions and control	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Mini 2	Record manager	Structs and files	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Capstone	Console management system	Pointers, modules and debugging	Brief; working/source file or reproducible environment; output; validation notes; reflection.

Standard project rubric

Dimension	Weight	What earns credit
Problem framing	15%	Clear objective, user/business need, assumptions and success criteria.
Technical/design accuracy	25%	Correct methods, appropriate tools and sound decisions.
Completeness	15%	Required features, assets, data or workflow are present.
Testing and validation	15%	Important normal, edge, quality or failure cases are checked.
Evidence and reproducibility	15%	Source/working files, setup, inputs, outputs and version information are available.
Documentation	5%	Concise structure, instructions and limitations.
Viva / ownership	10%	Learner can explain decisions, demonstrate work and respond to a change request.

Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.

NT-C-016 | C Programming | Career Roles, Evidence & Gap Actions

Explainable top-three occupation matching

Rank	Occupation	Default weighted skills	Minimum evidence	Gap action
1	C Programming Trainee	Syntax 34% [critical]; functions 33% [critical]; files 33%	Working C portfolio	If a critical skill is below 60 or working c portfolio is missing, return Improve First or Needs Evidence with the linked module/project.
2	Embedded Software Trainee - Foundation	Pointers 34% [critical]; memory 33% [critical]; debugging 33%	Low-level exercises	If a critical skill is below 60 or low-level exercises is missing, return Improve First or Needs Evidence with the linked module/project.
3	Junior Systems Programming Trainee	Modular C 50% [critical]; diagnostics 50% [critical]	Capstone source code	If a critical skill is below 60 or capstone source code is missing, return Improve First or Needs Evidence with the linked module/project.

Course readiness	45% section mocks + 20% mini projects + 25% capstone + 10% viva.
Role readiness	45% role-relevant mastery + 25% projects/capstone + 20% aligned mocks + 10% verified resume evidence.
Evidence confidence	Show separately; Apply Now requires at least 70.
Resume extraction	Use only relevant skills, projects, education, experience and portfolio evidence; minimise personal data before external model processing.

A student may be Apply Now for one mapped occupation and Improve First for another. Return independent evidence and gaps for each role.

NT-C-016 | C Programming | FAQs, Misconceptions & LLM Response Pack

Course-specific retrieval and response patterns

Approved FAQ	Answer
Who is C Programming for?	Beginners needing procedural programming and memory fundamentals.
What should I know before starting?	Basic computer use and logical thinking.
How long is the approved learning journey?	6-8 weeks / 60-90 guided hours.
What will I be able to demonstrate?	Develop and debug modular C programs using arrays, pointers, structures and files.
Does completion guarantee a job?	No. Completion and role readiness are evidence-based guidance; employment, placement, salary and employer selection are not guaranteed.
What if I fail a section?	CareerTours identifies the failed skill tags and module, assigns targeted practice, and permits an equivalent retest after remediation.

Misconception	Approved correction
Attendance proves mastery	False - require tests and observable practical/project evidence.
One total score proves every role fit	False - calculate each occupation using its weighted skills and evidence.
The LLM can add missing details	False - retrieve approved records or state that evidence is insufficient.
Course completion guarantees employment	False - never claim guaranteed job, placement, salary or interview.
C Programming knowledge never changes	False - version-sensitive tools, laws, taxes, security and platform procedures require review.

Retrieval metadata

```
course_code=NT-C-016; corpus_version=3.0;
record_types=course_profile,module,section,skill,assessment_sample,project,occupation_mapping,faq,misconception;
approval_status=approved; effective_date=2026-08-12
```